CampusEats — External SOAP Partner Integration at the Contract Level

CS543 Web Services • Assignment 3 • 13 Oct 2026 • Tutorial 3 method

TeamID: MCA-CS543-G07 | **Repository:** campuseats / Ecommerce-1 (bags store) | **Stack:** MongoDB • Express • React • Node • Razorpay → SecurePay SOAP

Roll / Name	Assignment 2 role	Assignment 3 responsibility
Abhishek	Auth + Payment	WSDL authoring, fault mapping
Binamra Rana	Auth + Payment	SOAP envelopes, HTTP binding
Navendu	Auth + Order	Discovery / registry entry
Shruti	Catalogue + Order	Documentation (integration.pdf)
Yatindra	Catalogue + Order + Payment	Verification & packaging

Figure 1 — Team composition. Repo name is *Ecommerce-1*; brief still refers to it as *CampusEats* (bags for college students). Behaviour and services are identical.

Note on naming: the deployed folder is **Ecommerce-1** (bags store). The assignment sheet still calls it **CampusEats** — the brief text even notes: *'Naam CampusEats hai kyunki pehle idea canteen ka tha, ab ye campus ke liye bags bechta hai'*. All references below use the assignment name CampusEats for compliance; the artefact on disk is Ecommerce-1.

1 Context — Partner edge and why SOAP

Chosen partner: SecurePay Gateway — external payment gateway (alternative: logistics/SMS; payment chosen because it is the only edge with money at risk). **Operation integrated:** `charge(orderRef, amount, currency, cardToken, studentEmail)` → `ChargeResponse(transactionId, status, amountCharged, authCode, gatewayRef)` with `PaymentFault(code, reason, retryable)`. WSDL service is `PaymentGatewayService` at `https://api.securepay.example.com/soap/PaymentGateway`.

Why this edge is SOAP while the rest of CampusEats is REST

1. **Strong contract.** The WSDL freezes types, required fields and the fault schema before any code runs — CampusEats and the bank cannot drift. REST with JSON would need out-of-band coordination.
2. **Message-level security.** Credentials travel in the SOAP `Header` with `mustUnderstand` and a signature point, so they survive intermediaries. The catalogue/cart REST surface needs only transport TLS.
3. **Transactions.** The `charge` is atomic and idempotent via `gatewayRef / authCode` stored exactly as Assignment 2's `payments` table expects (razorpay_order_id / payment_id / status). ACID-style assurance outweighs REST's cacheability only on this money edge.
4. **Rest stays REST** because product search, cart and catalogue are resource-oriented, cacheable and human-driven; payment is the sole partner edge where a frozen contract and WS-* guarantees justify SOAP.

The REST surface from Assignment 2 is unchanged: `GET /api/products`, `GET /api/products/:id`, `POST /api/payment/order`, `POST /api/payment/verify`. The SOAP partner is consumed only by the Payment Service at checkout and never exposed to the student.

2 Partner WSDL — six elements (partner.wsdl)

File `partner.wsdl` is a complete, valid WSDL 1.1 (document/literal) — the contract CampusEats binds to. It is **authored**, not generated, per Tutorial 3.

#	WSDL element	What it contains in this contract
1	types	XSD schema tns: AuthHeader (merchantId, apiKey, timestamp, nonce) + Charge + ChargeResponse + PaymentFault
2	message	AuthHeaderMsg, ChargeRequestMsg, ChargeResponseMsg, PaymentFaultMsg (each wraps one element)
3	portType	PaymentPortType with one operation **charge** (input → output, fault PaymentFault)
4	binding	PaymentBinding — `soap:binding style=document transport=soap/http`; soapAction `http://securepay.example.com/payment/charge`
5	service	PaymentGatewayService (documentation: SecurePay SOAP Payment Gateway)
6	port + address	PaymentGatewayPort → `https://api.securepay.example.com/soap/PaymentGateway`

Table 1 — Six WSDL elements and their content.

WSDL excerpt — portType + binding (for viva):

```
<portType name="PaymentPortType">
  <operation name="charge">
    <input message="tns:ChargeRequestMsg"/>
    <output message="tns:ChargeResponseMsg"/>
    <fault name="PaymentFault" message="tns:PaymentFaultMsg"/>
  </operation>
</portType>
<binding name="PaymentBinding" type="tns:PaymentPortType">
  <soap:binding style="document" transport="http://schemas.xmlsoap.org/soap/http"/>
  <operation name="charge">
    <soap:operation soapAction="http://securepay.example.com/payment/charge"/>
    <input>
      <soap:header message="tns:AuthHeaderMsg" part="auth" use="literal"/>
      <soap:body parts="parameters" use="literal"/>
    </input>
    <output><soap:body use="literal"/></output>
    <fault name="PaymentFault"><soap:fault name="PaymentFault" use="literal"/></fault>
  </operation>
</binding>
<service name="PaymentGatewayService">
  <port name="PaymentGatewayPort" binding="tns:PaymentBinding">
    <soap:address location="https://api.securepay.example.com/soap/PaymentGateway"/>
  </port>
</service>
```

3 SOAP messages by hand (Tasks 3) — element-for-element match

All three envelopes use `http://securepay.example.com/payment` as targetNamespace, match the XSD element names, and validate against the WSDL types. Only design-time — no server.

3.1 Request — credentials in Header, call in Body (soap-request.xml)

```
<soap:Envelope xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/"
               xmlns:pay="http://securepay.example.com/payment">
  <soap:Header>
    <pay:AuthHeader soap:mustUnderstand="1">
      <pay:merchantId>CAMPUSEATS_MERCH_8421</pay:merchantId>
      <pay:apiKey>sp_test_9f3k2pLq8xW1vT4yZ</pay:apiKey>
      <pay:timestamp>2026-10-13T17:30:00Z</pay:timestamp>
      <pay:nonce>6f8a9c1e-4b2d-4f0a-9c3e-1a2b3c4d5e6f</pay:nonce>
    </pay:AuthHeader>
  </soap:Header>
  <soap:Body>
    <pay:Charge>
      <pay:orderRef>ORD_20261013_001</pay:orderRef>
      <pay:amount>2698.00</pay:amount>
      <pay:currency>INR</pay:currency>
      <pay:cardToken>tok_visa_4242_securepay</pay:cardToken>
      <pay:studentEmail>abhishek.student@campuseats.example.com</pay:studentEmail>
    </pay:Charge>
  </soap:Body>
</soap:Envelope>
```

3.2 Success response (soap-response.xml)

```
<soap:Envelope xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/"
               xmlns:pay="http://securepay.example.com/payment">
  <soap:Body>
    <pay:ChargeResponse>
      <pay:transactionId>txn_sp_8a7f3c9d21e44b9a8c1d0e2f</pay:transactionId>
      <pay:status>APPROVED</pay:status>
      <pay:amountCharged>2698.00</pay:amountCharged>
      <pay:authCode>AUTH_482917</pay:authCode>
      <pay:gatewayRef>gw_ref_20261013_001</pay:gatewayRef>
    </pay:ChargeResponse>
  </soap:Body>
</soap:Envelope>
```

3.3 Fault — realistic failure (soap-fault.xml)

```
<soap:Envelope xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/"
               xmlns:pay="http://securepay.example.com/payment">
  <soap:Body>
    <soap:Fault>
      <faultcode>soap:Server</faultcode>
      <faultstring>Payment declined by issuer</faultstring>
      <detail>
        <pay:PaymentFault>
          <pay:code>card_declined</pay:code>
          <pay:reason>Insufficient funds / card limit exceeded for order ORD_20261013_001</pay:reason>
          <pay:retryable>false</pay:retryable>
          <pay:orderRef>ORD_20261013_001</pay:orderRef>
        </pay:PaymentFault>
      </detail>
    </soap:Fault>
  </soap:Body>
</soap:Envelope>
```

Matching note: `AuthHeader` is declared as `soap:header` in the binding, so it appears in `soap:Header` (not in the Body). The fault `detail` carries the typed `PaymentFault` element exactly as defined in `types` — not a plain string.

4 HTTP binding — POST that carries the request (Task 4)

The WSDL declares `soap:binding style='document' transport='http://schemas.xmlsoap.org/soap/http'` (SOAP over HTTP). The consumer therefore sends one **HTTP POST** to the service/port endpoint:

```
POST /soap/PaymentGateway HTTP/1.1
Host: api.securepay.example.com
Content-Type: text/xml; charset=utf-8
Content-Length: 892
SOAPAction: "http://securepay.example.com/payment/charge"
Connection: keep-alive

<?xml version="1.0" encoding="UTF-8"?>
<soap:Envelope ...> ... as in soap-request.xml ... </soap:Envelope>
```

Key: `SOAPAction` is taken verbatim from `binding/operation/soap:operation/@soapAction` in partner.wsdl. `Content-Type: text/xml; charset=utf-8` is SOAP 1.1 (SOAP 1.2 would be `application/soap+xml`). Endpoint is `service/port/soap:address/@location`. Success: `200 OK` + SOAP envelope; Fault: `500 Internal Server Error` + `soap:Fault`.

5 Discovery — modern form (Task 5)

No live UDDI server is run (per brief). Discovery is design-time in two equivalent modern forms:

(a) Direct URL (contract-first download): the gateway publishes its WSDL at a stable URL that CampusEats binds to at build time:

```
WSDL URL:  https://api.securepay.example.com/soap/PaymentGateway?wsdl
Endpoint:  https://api.securepay.example.com/soap/PaymentGateway
```

(b) Catalogue / registry lookup (tModel-style pointer): in production CampusEats does not hard-code the WSDL. An internal **Service Catalogue** (e.g. Backstage / API gateway registry / WSO2 Governance Registry) holds one record that points to that WSDL as a `tModel`. The developer searches the catalogue for 'SecurePay' and receives the WSDL location plus the `SOAPAction`. This satisfies the 'modern UDDI' requirement without running a UDDI node.

One-record catalogue / registry entry (JSON — tModel style)

```
{
  "businessEntity": {
    "name": "SecurePay Technologies Pvt. Ltd.",
    "description": "PCI-DSS certified payment gateway partner for CampusEats",
    "contact": "partner-support@securepay.example.com"
  },
  "businessService": {
    "serviceKey": "uddi:securepay.example.com:service:payment-gateway",
    "name": "SecurePay PaymentGatewayService",
    "description": "SOAP service for charging student card tokens (CampusEats edge)"
  },
  "bindingTemplate": {
    "bindingKey": "uddi:securepay.example.com:binding:payment-soap-http",
    "accessPoint": "https://api.securepay.example.com/soap/PaymentGateway",
    "accessPointType": "https",
    "bindingDetail": "SOAP 1.1 over HTTP, document/literal"
  },
  "tModel": {
    "tModelKey": "uddi:securepay.example.com:tmodel:payment-wsdl",
    "name": "SecurePay Payment WSDL",
    "overviewURL": "https://api.securepay.example.com/soap/PaymentGateway?wsdl",
    "overviewDoc": "WSDL defining portType PaymentPortType with operation charge; binding PaymentBinding (SOAP/HT
    "categoryBag": ["uddi:types:wsdl", "protocol:soap-http", "domain:payment"]
  }
}
```


This single record lets CampusEats discover: **who** (business), **what** (service + operation `charge`), **where** (endpoint), and **how** (WSDL / tModel). The WSDL is the contract bound at compile time.

6 Fault mapping — partner vocabulary does not leak (Task 6)

CampusEats' Assignment 2 contract (`schema.sql` — `orders.status = PENDING_PAYMENT | PAID | … | CANCELLED`, `payments.status = CREATED | PAID | FAILED`) promises a **single student-facing error** for checkout. The SOAP partner's fine-grained fault is **translated at the Payment Service boundary**:

SecurePay PaymentFault.code	reason / retryable	CampusEats A2 mapping
card_declined	Insufficient funds, retryable=false	400 { success:false, message:"payment declined" } payments.status=FAILED, orders.status=CANCELLED
invalid_token	Token expired, retryable=false	400 { success:false, message:"payment declined" } ask student to re-enter card
gateway_timeout	Upstream timeout, retryable=true	503 { success:false, message:"payment unavailable, try again" } keep PENDING_PAYMENT, idempotent retry with orderRef+nonce
duplicate_charge	orderRef already charged	409 { success:false, message:"payment already processed" } return existing transactionId idempotently

Table 2 — Fault translation. Partner strings never reach the frontend.

Rule: The strings `card_declined`, `gateway_timeout`, `PaymentFault` and the SOAP `faultcode` **never reach the frontend**. The student sees only Assignment 2's vocabulary: **'payment declined'** (do not retry) or **'payment unavailable, try again'** (may retry). Internally the Payment Service logs `transactionId / gatewayRef / authCode` on success and `code+reason` on fault, then updates the local `payments` row — preserving the A2 invariant that every table belongs to one service and cross-service refs are plain IDs.

Submission checklist

✓ `integration.pdf` (this document) — Tasks 1, 4, 5, 6 + TeamID / Roll / Name ✓ `partner.wsdl` (Task 2, six elements) ✓ `soap-request.xml`, `soap-response.xml`, `soap-fault.xml` (Task 3) ✓ `Assignment-3.zip` (one Zip with the five files, uploaded by Team Leader). All artefacts are **design-time only** — no running SOAP server or live gateway.